考察的一些细节

数码管显示模式切换

在考试中，我们经常会遇到数码管显示模式的切换。这部分比较简单，只需使用一个变量（通常是`Seg_Show_Mode`）来控制不同的显示模式。在按键处理中这样实现：

```
// 按键4处理(切换显示模式)
if (Key_Down == 4)
{
    Seg_Show_Mode = (++Seg_Show_Mode) % 5; // 循环切换5种显示模式
}
```

参数设置的两种情况

情况一：退出后生效

当题目中提到"退出后生效"或"设置最大最小值后进行校验"时，我们需要使用两个变量：

- 一个真实值：用于系统判断和实际功能
- 一个控制值：用于在参数界面显示和修改

切换界面时，需要在这两个值之间进行传递：

```
idata unsigned char Ad_Para;      // 真实值，后续做对比的时候用到
idata unsigned char Ad_Para_Ctrl; // 控制值，退出控制界面的时候赋值给真实值
// 按键4处理(切换显示模式)
if (Key_Down == 4)
{
    if (Seg_Show_Mode == 0)
        Ad_Para_Ctrl = Ad_Para; // 将参数真实值给控制值，界面1为参数更改界面
    else if (Seg_Show_Mode == 1)
        Ad_Para = Ad_Para_Ctrl; // 将控制值给参数真实值，退出参数更改界面
    Seg_Show_Mode = (++Seg_Show_Mode) % 5; // 循环切换5种显示模式
}
```

情况二：实时生效

如果题目没有提到"退出后生效"，则表示参数需要实时更新，此时不需要控制值，直接修改真实值即可。

参数修改的边界处理

通常题目会指定参数的修改方式。如果没有明确说明，默认采用"锁死上下限"方式。

假设`Ad_Para`的范围是0~10，步进为1，两种常见处理方式：

循环模式（溢出后回到另一端）

```
// 下限到了就变成上限，上限到了就会变成下限
// 0 1 2 3 4 5 6 7 8 9 10 0 1 2 3 ...
if (Key_Down == 5)
    Ad_Para_Ctrl = (Ad_Para_Ctrl == 10) ? 0 : Ad_Para_Ctrl + 1;
if (Key_Down == 6)
    Ad_Para_Ctrl = (Ad_Para_Ctrl == 0) ? 10 : Ad_Para_Ctrl - 1;
```

锁死模式（到达边界后不再变化）

```
// 锁死上下限
if (Key_Down == 5)
    Ad_Para_Ctrl = (Ad_Para_Ctrl == 10) ? 10 : Ad_Para_Ctrl + 1;
if (Key_Down == 6)
    Ad_Para_Ctrl = (Ad_Para_Ctrl == 0) ? 0 : Ad_Para_Ctrl - 1;
```

数据计算的类型转换注意事项

题目中常常会给出一个分段函数关系（如频率转湿度）。计算过程中一定要注意数据类型，尤其是涉及除法时。由于我们经常使用`unsigned int`或`unsigned char`定义变量，但进行除法时需要进行类型转换，避免整数除法带来的精度损失。

例如，对于下面的转换关系：

- 当Freq > 20000时，Hum = 100
- 当Freq < 2000时，Hum = 10
- 中间区域：Hum = (Freq - 2000) / 200 - 10

正确的实现方式应该是：

```
if (Freq < 2000)
    Hum = 10;
else if (Freq > 20000)
    Hum = 100;
else
    Hum = (float)(Freq - 2000) / 200.0f - 10;
```

注意`(float)`强制类型转换的使用，避免整数除法导致的精度问题。

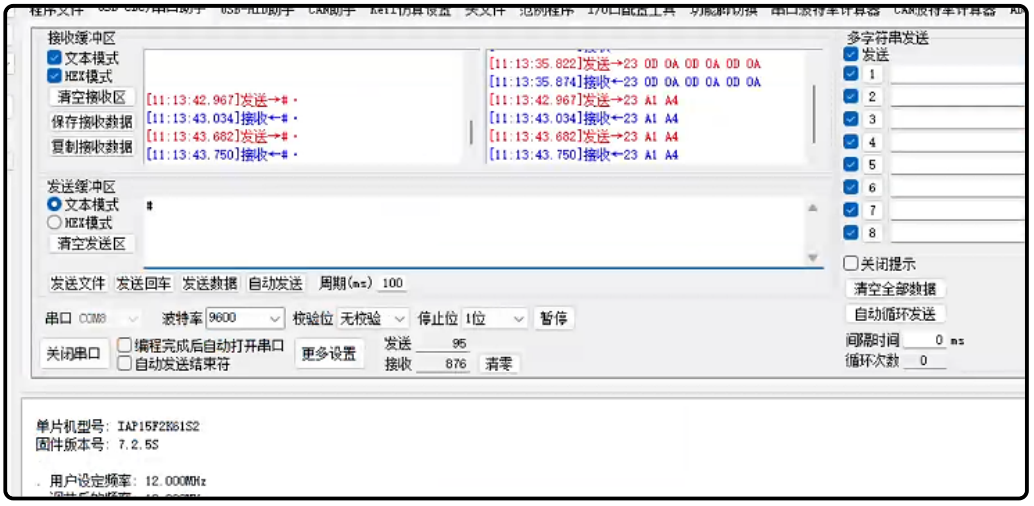
还有就是关于第十四届国赛的超声波的计算，一定要注意运算范围，因为本质上他运算的时候不会自动帮你提升你的变量的范围，所以一旦超过变量范围就会爆掉`distance = ((float)(CH << 8 | CL)/ 20000.0f) * speed`才是正确的书写方式

并且注意，通常来说，我们的unsigned int 和int不能混合运算，尤其是涉及到了负数的情况，我们需要取反或者使用abs函数，通常来说最简单的方法就是取反就好了，比如第十五届省赛，我么数码管显示可以这样写

```c
if (Calibration_Para_Ctrl < 0)
{
    Seg_Buf[4] = 13; //-
    Seg_Buf[5] = (-Calibration_Para_Ctrl) / 100;
    Seg_Buf[6] = 0;
    Seg_Buf[7] = 0;
}
else if (Calibration_Para_Ctrl == 0)
{
    Seg_Buf[4] = 10;
    Seg_Buf[5] = 10;
    Seg_Buf[6] = 10;
    Seg_Buf[7] = 0;
}
else
{
    Seg_Buf[4] = 10;
    Seg_Buf[5] = Calibration_Para_Ctrl / 100;
    Seg_Buf[6] = 0;
    Seg_Buf[7] = 0;
}
```

串口调试注意事项

在串口通信调试中，建议在超时解析处理中先使用`printf`打印接收到的数据，确认数据接收正常后再进行后续处理。



注意事项：

1. 如果题目没有要求发送`\r\n`，不要自作主张添加
2. 检查返回的数据是否包含`0D 0A`（回车换行符）
3. 在ISP中，发送`\r\n`只需直接按回车键即可

```c
void Uart_Proc()
{
    // 如果没有接收到数据则返回
    if (Uart_Rx_Index == 0)
        return;
    // 检查是否接收超时(10ms)
    if (Uart_Rx_Tick >= 10)
    {
        Uart_Rx_Flag = 0;
        Uart_Rx_Tick = 0;
        printf("%s", Uart_Rx_Buf);
        // 清空接收缓冲区，准备下一次接收
        memset(Uart_Rx_Buf, 0, Uart_Rx_Index);
        Uart_Rx_Index = 0;
    }
}
```

学员遇见的问题

按键相关问题

按键无响应/失灵

- P3口使用`==`错误：应使用单等号`=`赋值而非双等号`==`判断
- P34和信号线接通：需拔掉跳线帽
- 多个功能使用同一定时器：长按短按共用定时器导致功能冲突
- 按键底层缺少临时变量初始化：`temp = 0`未设置

按键重复触发

- 按键处理中缺少临时变量清零操作

长按短按问题

- 长按短按共用变量：共用计时导致功能异常
- 错误使用触发方式：长短按判断应使用`Key_Up`而非`Key_Down`

- 计时范围未限定：上电开始就计数导致按键异常

数码管显示问题

数码管不显示/闪烁

- 定时器使用错误：未设置自动重装或初始化位置不对
- 减速逻辑错误：未加入减速导致扫描过快无法看到显示
- P2口操作错误：`P2 = P2 & 0x1f | 0xe0` 与 `P2 &= 0x1f | 0xe0` 不同
- 小数点显示问题：应使用`P2 &= 0x7f`而非`P2 |= 0x7f`

数值显示异常

- 负数显示：需单独处理符号位，使用`abs`函数处理数值部分，或者判断一下是否是负数然后进行运算
- 数据溢出：注意数据类型范围，`unsigned char`仅为0-255

定时器问题

定时器异常

- 定时器初始化位置错误：应放在while循环外
- 未设置自动重装：导致定时不准确
- 定时器冲突：不同功能共用一个定时器（超声波建议直接使用PCA）
- 没有开启全局中断：ET0/ET1/EA等中断控制位未开启

I2C/AD转换相关问题

AD读取异常

- 连续读取不稳定：连续读取需添加延时（如`I2C_Delay(255)`）
- 上电瞬间不稳定：需在while前空读一次并添加适当延时

传感器问题

DS1302时钟问题

- 时钟跳变：写入时间前将秒寄存器CH位拉高停止计时
- 分钟跳变：读取时间的for循环应从0开始
- 时钟停止：检查uwTick是否为long类型（防止65s后溢出）【调度器】
- 闹钟触发后无法关闭：修改匹配逻辑，确保三位时间都判断完才触发

超声波问题

测距异常

- 测距为0：检查底层是否有语法错误（缺少分号等）
- 数值乱跳：调整Delay12us函数i的值（33~50）
- 数据异常：检查计算公式，如`distance = ((float)(CH << 8 | CL)/20000.0f) * speed`

串口通信问题

串口异常

- 乱码接收：检查晶振设置（应为12MHz）
- 无响应：检查接收缓冲区是否初始化（是否使用pdata/xdata进行定义的buf）
- 通信匹配失败：注意发送缓冲区可能含有换行符\r\n

EEPROM问题

EEPROM读写异常

- 上电读取为0：注意初始化顺序，定时器应最后初始化
- 写入后程序卡死：EEPROM写入之间添加延时，避免连续写入（10个Delay255）

代码编写和编译问题

编译错误

- idata堆栈溢出：确保data<210，将部分数据移至pdata区
- 宏定义嵌套报错：避免在.h和.c中重复引用同一头文件
- 函数命名不一致：确保函数调用名与定义名完全一致
- 数据计算不准确：添加类型转换，如`(float)`确保浮点计算精度
- 内存限制错误：检查Keil安装是否正确，或进行内存管理优化